

vLLM Fallback Fix - Implementation Plan

Executive Summary

The vLLM audit revealed that while vLLM is properly wired and CAN generate responses, the fallback chain is not executing when the primary provider fails. This document provides the detailed implementation plan to fix this critical issue.

Root Cause

The `LLMRouter.process_chat_request()` method has fallback logic, but it only yields text chunks without metadata. When a provider fails and falls back to vLLM/Transformers, the metadata still shows the requested provider instead of the actual provider used.

Additionally, the `LLMRouter.generate_with_degraded_runtime_fallback()` method exists and works (has tests), but is never called by the orchestrator.

Current Flow (BROKEN)

```
API Route (runtime.py:414)
  → LangGraphOrchestrator.process()
  → Graph Node: router_select
    → LLMRouter.select_provider() - selects Gemini
  → Graph Node: response_synth
    → LLMRouter.process_chat_request()
      → Tries Gemini
      → Gemini fails
      → Returns static degraded message
      → NO fallback to vLLM/Transformers
  → Metadata shows: provider="gemini" (WRONG)
```

PROF

Target Flow (FIXED)

```
API Route (runtime.py:414)
  → LangGraphOrchestrator.process()
  → Graph Node: router_select
    → LLMRouter.select_provider() - selects Gemini
  → Graph Node: response_synth
    → Try primary provider (Gemini)
    → Catch failure
    → Call LLMRouter.generate_with_degraded_runtime_fallback()
      → Tries builtin_vllm
      → vLLM succeeds
      → Returns content + metadata
  → Metadata shows:
    - requested_provider="gemini"
```

```
- actual_provider="vllm"
- runtime_engine="vllm"
- response_source="live_model"
- degraded_mode=true
- fallback_level=1
```

Implementation Steps

Step 1: Modify response_synth.py to Use Fallback Method

File: `src/ai_karen_engine/core/langgraph_orchestrator/nodes/response_synth.py`

Changes:

1. Import `generate_with_degraded_runtime_fallback`
2. Wrap `process_chat_request` in try/catch
3. On failure, call `generate_with_degraded_runtime_fallback`
4. Store metadata in state for API route to return

Code Location: Lines 70-108

Current Code:

```
if self._llm_router and (tool_results or reasoning_result):
    try:
        selection = await self._llm_router.select_provider(...)
        if selection:
            provider_name, model_name = selection
            response_gen = self._llm_router.process_chat_request(...)
            final_text = ""
            async for chunk in response_gen:
                final_text += chunk
            if final_text.strip():
                return final_text.strip()
    except Exception as e:
        logger.warning(f"LLM-based synthesis failed: {e}")
```

New Code:

```
if self._llm_router and (tool_results or reasoning_result):
    try:
        selection = await self._llm_router.select_provider(...)
        if selection:
            provider_name, model_name = selection

            try:
                # Try primary provider
                response_gen =
```

```

self._llm_router.process_chat_request(...)
    final_text = ""
    async for chunk in response_gen:
        final_text += chunk

    if final_text.strip():
        # Store metadata for successful primary provider
        state["llm_metadata"] = {
            "requested_provider": provider_name,
            "actual_provider": provider_name,
            "requested_model": model_name,
            "actual_model": model_name,
            "runtime_engine": provider_name,
            "response_source": "live_model",
            "degraded_mode": False,
            "used_fallback": False,
        }
        return final_text.strip()

    except Exception as provider_error:
        # Primary provider failed - try fallback chain
        logger.warning(
            f"Primary provider {provider_name} failed:
{provider_error}. "
            "Attempting degraded runtime fallback."
        )

        fallback_result = await
self._llm_router.generate_with_degraded_runtime_fallback(
    request=ChatRequest(message=synthesis_prompt,
stream=False),
    requested_provider=provider_name,
    requested_model=model_name,
    failure_reason=str(provider_error),
)

        if fallback_result and fallback_result.get("content"):
            # Store fallback metadata
            state["llm_metadata"] =
fallback_result.get("metadata", {}).get("llm", {})
            return fallback_result["content"]

        # Fallback also failed
        logger.error("All providers failed including fallback
chain")

    except Exception as e:
        logger.warning(f"LLM-based synthesis failed: {e}")

```

PROF

Step 2: Update API Route to Return Metadata

File: `src/ai_karen_engine/api_routes/chat/runtime.py`

Changes:

1. Extract `llm_metadata` from `final_state`
2. Merge into `response_metadata`
3. Ensure UI receives actual provider info

Code Location: Lines 540-560

Add After Line 550:

```
# Extract LLM metadata from orchestrator state
llm_metadata = final_state.get("llm_metadata", {})
if llm_metadata:
    response_metadata.update({
        "requested_provider": llm_metadata.get("requested_provider"),
        "actual_provider": llm_metadata.get("actual_provider"),
        "requested_model": llm_metadata.get("requested_model"),
        "actual_model": llm_metadata.get("actual_model"),
        "runtime_engine": llm_metadata.get("runtime_engine"),
        "response_source": llm_metadata.get("response_source"),
        "fallback_level": llm_metadata.get("fallback_level", 0),
        "fallback_chain": llm_metadata.get("fallback_chain", []),
        "attempted_providers": llm_metadata.get("attempted_providers",
    []),
    })
```

Step 3: Add Logging to VLLMRuntime Internal Fallback

File: `src/ai_karen_engine/inference/vllm_runtime.py`

Changes:

1. Add structured logging when vLLM falls back to Transformers
2. Log provider selection attempts
3. Log fallback chain execution

Code Location: Lines 20-131 (VLLMRuntime class)

Add Logging:

```
logger.info(
    "vLLM internal fallback triggered",
    extra={
        "from_provider": "vllm",
        "to_provider": "transformers",
        "reason": "vllm_unavailable",
```

```
}  
)
```

Step 4: Create vLLM Diagnostics Endpoint

File: `src/ai_karen_engine/api_routes/health/providers.py` (NEW)

Content:

```
"""Provider health and diagnostics endpoints."""  
  
import logging  
from typing import Dict, Any  
from fastapi import APIRouter, HTTPException  
import httpx  
  
logger = logging.getLogger(__name__)  
router = APIRouter(prefix="/api/health/providers", tags=["health",  
"providers"])  
  
@router.get("/vllm")  
async def get_vllm_health() -> Dict[str, Any]:  
    """  
    Get vLLM provider health and diagnostics.  
  
    Tests:  
    - vLLM server connectivity  
    - Model availability  
    - Generation capability  
    """  
    from ai_karen_engine.config.llm_provider_config import  
    get_provider_registry  
  
    registry = get_provider_registry()  
    vllm_info = registry.get_provider_info("builtin_vllm")  
  
    if not vllm_info:  
        return {  
            "provider": "vllm",  
            "enabled": False,  
            "healthy": False,  
            "error": "vLLM not configured in provider registry"  
        }  
  
    base_url = vllm_info.get("base_url", "http://localhost:8001/v1")  
    health_url = vllm_info.get("health_check_url",  
"http://localhost:8001/health")  
  
    result = {
```

```

        "provider": "vllm",
        "enabled": vllm_info.get("enabled", False),
        "base_url": base_url,
        "health_check_url": health_url,
        "default_model": vllm_info.get("default_model"),
    }

# Test health endpoint
try:
    async with httpx.AsyncClient(timeout=5.0) as client:
        health_response = await client.get(health_url)
        result["health_endpoint_ok"] = health_response.status_code
    == 200
except Exception as e:
    result["health_endpoint_ok"] = False
    result["health_error"] = str(e)

# Test models endpoint
try:
    async with httpx.AsyncClient(timeout=5.0) as client:
        models_response = await client.get(f"{base_url}/models")
        if models_response.status_code == 200:
            models_data = models_response.json()
            result["models_endpoint_ok"] = True
            result["available_models"] = [
                m.get("id") for m in models_data.get("data", [])
            ]
        else:
            result["models_endpoint_ok"] = False
except Exception as e:
    result["models_endpoint_ok"] = False
    result["models_error"] = str(e)

# Test generation
if result.get("models_endpoint_ok") and
result.get("available_models"):
    try:
        test_model = result["available_models"][0]
        async with httpx.AsyncClient(timeout=10.0) as client:
            gen_response = await client.post(
                f"{base_url}/chat/completions",
                json={
                    "model": test_model,
                    "messages": [
                        {"role": "user", "content": "Say 'test' in
one word."}
                    ],
                    "max_tokens": 10,
                    "temperature": 0.1,
                }
            )
        if gen_response.status_code == 200:
            gen_data = gen_response.json()

```

```

        content = gen_data.get("choices", [{}])
[0].get("message", {}).get("content", "")
        result["generation_test_ok"] = bool(content.strip())
        result["generation_test_response"] = content.strip()
    else:
        result["generation_test_ok"] = False
except Exception as e:
    result["generation_test_ok"] = False
    result["generation_error"] = str(e)

# Overall health
result["healthy"] = (
    result.get("health_endpoint_ok", False)
    and result.get("models_endpoint_ok", False)
    and result.get("generation_test_ok", False)
)

return result

```

Step 5: Add Integration Tests

File: `tests/integration/test_fallback_chain.py` (NEW)

Content:

```

"""Integration tests for provider fallback chain."""

import pytest
from unittest.mock import AsyncMock, patch, MagicMock

class TestFallbackChain:
    """Test provider fallback chain execution."""

    @pytest.mark.asyncio
    async def test_gemini_to_vllm_fallback(self):
        """Test fallback from Gemini to vLLM when Gemini fails."""
        from ai_karen_engine.services.models.routing.llm_router_service
import (
    LLMRouter,
    ChatRequest,
)

        router = LLMRouter()

        # Mock Gemini to fail
        with patch.object(router.registry, "get_provider") as
mock_get_provider:
            # First call (Gemini) fails
            gemini_provider = AsyncMock()

```

```

        gemini_provider.generate_response.side_effect =
Exception("Gemini unavailable")

        # Second call (vLLM) succeeds
        vllm_provider = AsyncMock()
        vllm_provider.generate_response.return_value = "vLLM
response"

        mock_get_provider.side_effect = [gemini_provider,
vllm_provider]

        # Execute fallback
        result = await
router.generate_with_degraded_runtime_fallback(
            request=ChatRequest(message="test"),
            requested_provider="gemini",
            requested_model="gemini-2.5-flash",
            failure_reason="Gemini unavailable",
        )

        # Verify fallback executed
        assert result["content"] == "vLLM response"
        assert result["metadata"]["llm"]["requested_provider"] ==
"gemini"
        assert result["metadata"]["llm"]["provider"] ==
"builtin_vllm"
        assert result["metadata"]["llm"]["response_source"] ==
"runtime_fallback"
        assert result["metadata"]["llm"]["degraded_mode"] is True

    @pytest.mark.asyncio
    async def test_metadata_reflects_actual_provider(self):
        """Test that metadata shows actual provider used, not
requested."""
        # Similar test verifying metadata accuracy
        pass

    @pytest.mark.asyncio
    async def test_vllm_to_transformers_fallback(self):
        """Test fallback from vLLM to Transformers when vLLM fails."""
        # Test internal vLLM fallback
        pass

```

PROF

Testing Plan

Manual Testing

1. Test Gemini → vLLM Fallback:

```
# Disable Gemini API key
unset GEMINI_API_KEY

# Make chat request
curl -sS http://localhost:8000/api/chat/chat \
  -H "Content-Type: application/json" \
  -d '{
    "message": "Hello",
    "provider": "gemini",
    "model": "gemini-2.5-flash"
  }' | jq '.metadata'

# Expected metadata:
# {
#   "requested_provider": "gemini",
#   "actual_provider": "vllm",
#   "runtime_engine": "vllm",
#   "response_source": "live_model",
#   "degraded_mode": true,
#   "fallback_level": 1
# }
```

2. Test vLLM Direct:

```
curl -sS http://localhost:8000/api/chat/chat \
  -H "Content-Type: application/json" \
  -d '{
    "message": "Hello",
    "provider": "vllm"
  }' | jq '.metadata'

# Expected metadata:
# {
#   "requested_provider": "vllm",
#   "actual_provider": "vllm",
#   "runtime_engine": "vllm",
#   "response_source": "live_model",
#   "degraded_mode": false
# }
```

3. Test Diagnostics:

```
curl -sS http://localhost:8000/api/health/providers/vllm | jq
```

Automated Testing

```
# Run integration tests
pytest tests/integration/test_fallback_chain.py -v

# Run vLLM smoke tests
KAREN_RUN_VLLM_SMOKE=1 \
KAREN_VLLM_BASE_URL=http://localhost:8001/v1 \
pytest tests/integration/test_vllm_smoke.py -v
```

Rollout Plan

Phase 1: Core Fix (CRITICAL)

- ☐ Modify response_synth.py to use fallback method
- ☐ Update API route to return metadata
- ☐ Test manually with Gemini → vLLM fallback
- ☐ Verify metadata accuracy

Phase 2: Observability

- ☐ Add logging to VLLMRuntime
- ☐ Create diagnostics endpoint
- ☐ Test diagnostics endpoint

Phase 3: Testing

- ☐ Add integration tests
- ☐ Run full test suite
- ☐ Update documentation

Phase 4: UI Fix

- ☐ Rebuild UI (Ollama fix)
- ☐ Test Ollama provider selection
- ☐ Verify no legacy error

Success Criteria

- ✓ When Gemini fails, system falls back to vLLM automatically
- ✓ Metadata shows actual_provider="vllm", not "gemini"
- ✓ response_source="live_model", not "emergency_static"
- ✓ Degraded mode flag is set correctly
- ✓ Fallback chain is logged
- ✓ Diagnostics endpoint works
- ✓ Integration tests pass
- ✓ UI shows correct provider info

Risks and Mitigation

Risk: Breaking existing functionality

Mitigation: Comprehensive testing, gradual rollout

Risk: Performance impact from fallback attempts

Mitigation: Timeout configuration, circuit breaker pattern

Risk: Metadata bloat

Mitigation: Only include essential fields, compress if needed

Timeline

- **Phase 1:** 2-4 hours (core fix)
- **Phase 2:** 1-2 hours (observability)
- **Phase 3:** 2-3 hours (testing)
- **Phase 4:** 30 minutes (UI rebuild)

Total: 6-10 hours

Next Steps

1. User confirms approach
2. Implement Phase 1 (core fix)
3. Test manually
4. Implement remaining phases
5. Full system test
6. Deploy

Status: Ready for implementation

Priority: CRITICAL

Assigned: Development team

Reviewed: Audit complete